

Sumário

1	Básico	1
1.1	Código padrão (template)	1
1.2	Código padrão com casos teste (template)	1
2	Fluxo e Emparelhamento	1
2.1	Dinic (fluxo máximo)	1
2.2	Fluxo máximo com custo mínimo . . .	1
2.3	Emparelhamento Húngaro	1
2.4	Kuhn-Munkres (KM)	1
3	Estruturas de Dados	1
3.1	PBDS (Set Indexado)	1
3.2	BIT (Fenwick Tree)	1
3.3	BIT 2D	2
3.4	Union-Find (DSU) .	2
3.5	Union-Find persistente	2
3.6	Tabela esparsa (RMQ em $O(1)$)	2
3.7	Segment Tree . . .	2
3.8	Segment Tree dinâmica	2
3.9	Segment Tree dinâmica 2D . . .	2
3.10	Segment Tree persistente	2
3.11	Segment Tree temporal	2
3.12	Treap (BST aleatória)	2
4	Grafos	2
4.1	Ordenação Topológica	2
4.2	Dijkstra	3
4.3	Bellman-Ford . . .	3
4.4	SPFA	3
4.5	Floyd-Warshall . .	3
4.6	Caminho Euleriano	4
4.7	Componentes Biconexos (BCC) . .	4
4.8	Componentes Fortemente Conexas (SCC)	4
4.9	Árvore Geradora Mínima (MST) . . .	4
4.10	Clique Maximal . .	4
4.11	Clique Máximo . .	4
4.12	Ciclo de Média Mínima	4
4.13	AGM Manhattan . .	4
5	Programação Dinâmica	4
5.1	PD em dígitos . .	4
5.2	SOS DP	4
6	Matemática	4
6.1	Fórmulas	4
6.2	Exponenciação Binária	5
6.3	Soma e multiplicação com <code>__int128</code>	5
6.4	Primos	5
6.5	Pares coprimos . .	5
6.6	Exponenciação rápida	5
6.7	Exponenciação rápida de matrizes .	5
6.8	Função totiente (ϕ)	5
6.9	Tabela de fatores	5
6.10	Números de Catalan	5
6.11	Teste de primalidade Miller-Rabin	5
6.12	Fatoração Pollard-Rho	5
6.13	Fatoração prima em $O(\log n)$	5
6.14	Multiplicação $O(1)$ com <code>__int128</code> . .	5
6.15	Problema de Josefo (Josephus)	5
6.16	Soma harmônica . .	5
6.17	Contagem de Pólya	5
6.18	FFT (Transformada Rápida de Fourier)	5

6.19	Jogo de Grundy . .	5
6.20	Algoritmo de Euclides Estendido .	5
6.21	Teorema Chinês do Resto (CRT) . . .	5
7	Strings	5
7.1	Arranjo de Sufixos (SA)	5
7.2	KMP (Knuth-Morris-Pratt)	5
7.3	Hash simples . . .	5
7.4	Hash duplo	5
7.5	Rabin-Karp	5
7.6	Trie	6
7.7	Função Z	6
7.8	Rotação mínima . .	6
7.9	Manacher (palíndromos)	6
7.10	Árvore de palíndromos (PalTree) . .	6
7.11	Subsequências distintas	6
8	Árvores	6
8.1	LCA (Menor Ancestral Comum) . . .	6
8.2	Hash de árvore . .	6
8.3	Decomposição Pesada-Leve (HLD)	6
9	Geometria	6
9.1	Definições 2D . .	6
9.2	Definições 3D . .	6
9.3	Definição de reta	6
9.4	Operações básicas	6
9.5	Área do polígono .	6
9.6	Ponto dentro do polígono	6
9.7	Fecho convexo . .	6
9.8	Soma de Minkowski	6
9.9	Distância mais curta entre polígonos	6
9.10	Truque do fecho convexo (CHT) . .	6
9.11	Ordenação polar .	6
9.12	Teorema de Pick .	6
9.13	Par de pontos mais próximos	6
9.14	Par de pontos mais distantes	6
9.15	Mediana geométrica (Weiszfeld) . . .	6
9.16	Linha de varredura	6
9.17	Interseção círculo-polígono .	6
9.18	Tangentes a dois círculos	6
9.19	Interseção de dois círculos	6
9.20	Cobertura de círculo	6
9.21	Círculo mínimo envolvente	6
9.22	Esfera mínima envolvente	6
9.23	Retângulo envolvente de área mínima/máxima . . .	6
9.24	Interseção de semiplanos	7
9.25	União de polígonos	7
9.26	Cobertura de polígonos	7
9.27	Pontos notáveis do triângulo	7
10	Problemas Especiais	7
10.1	Contagem de substrings	7
10.2	Ordem parcial 3D	7
10.3	LCS cíclico . . .	7
10.4	Simulated Annealing (têmpera simulada)	7
10.5	Raiz quadrada discreta	7

1 Básico

1.1 Código padrão (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

#define fastio ios::sync_with_stdio(0); cin.tie(0)

int main() {
    fastio;

    // code

    return 0;
}
```

1.2 Código padrão com casos teste (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

#define fastio ios::sync_with_stdio(0); cin.tie(0)

void solve() {
    // code
}

int main() {
    fastio;

    int tc; cin >> tc;
    while (tc--) solve();

    return 0;
}
```

2 Fluxo e Emparelhamento

2.1 Dinic (fluxo máximo)

2.2 Fluxo máximo com custo mínimo

2.3 Emparelhamento Húngaro

2.4 Kuhn-Munkres (KM)

3 Estruturas de Dados

3.1 PBDS (Set Indexado)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define ordered_set tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>
using namespace __gnu_pbds;

// ordered_set s;
// s.insert(10); s.insert(20); s.insert(30);
// O(logN):
// s.find_by_order(1); // Retorna iterador para o 2º (s[i])
// s.order_of_key(30); // Retorna 2 (posição do valor X no set)
```

3.2 BIT (Fenwick Tree)

```
struct FenwickTree {
    vector<int> bit;
    int n;

    FenwickTree(int n) {
        this->n = n;
    }
}
```

```

    bit.assign(n, 0);
}

FenwickTree(vector<int> const &v) : FenwickTree(v.
    size()){
    for (int i = 0; i < v.size(); ++i)
        add(i, v[i]);
}

int soma(int r) {
    int ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1)
        ret += bit[r];
    return ret;
}

int soma(int l, int r) {
    return soma(r) - soma(l - 1);
}

void add(int idx, int delta) {
    for (; idx < n; idx = idx | (idx + 1))
        bit[idx] += delta;
}
};

```

3.3 BIT 2D

3.4 Union-Find (DSU)

```

//  $O\alpha(N) \approx O(1)$ 
vector<int> pai, tam;
void init(int n) {
    pai.resize(n+1);
    tam.assign(n+1, 1);
    iota(pai.begin(), pai.end(), 0); //  $pai[i] = i$ 
}

int find(int x) {
    if (x == pai[x]) return x;
    return pai[x] = find(pai[x]); // path compression
}

// Union by Size: anexa a árvore menor na maior
void join(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (tam[x] < tam[y]) swap(x, y);

    pai[y] = x;
    tam[x] += tam[y];
}

// Verifica se dois elementos estão no mesmo conjunto
bool mesmoConjunto(int x, int y) {
    return find(x) == find(y);
}

// Retorna o tamanho do componente que contém x
int tamanho(int x) {
    return tam[find(x)];
}

```

3.5 Union-Find persistente

3.6 Tabela esparsa (RMQ em $O(1)$)

3.7 Segment Tree

```

struct SegTree {
    vector<ll> vet, tree;
    int n;

    SegTree(int n) : n(n) {
        tree.resize(4 * n);
        vet.resize(n + 1);
    }
};

```

```

}

void build(int no, int l, int r) {
    if (l == r) {
        tree[no] = vet[l];
        return;
    }

    int m = l + (r - l) / 2;
    build(2 * no, l, m);
    build(2 * no + 1, m + 1, r);

    tree[no] = tree[2 * no] + tree[2 * no + 1];
}

void update(int no, int l, int r, int pos, ll val) {
    if (l == r) {
        tree[no] = val;
        return;
    }
    int m = l + (r - l) / 2;

    if (pos <= m) update(2 * no, l, m, pos, val);
    else
        update(2 * no + 1, m + 1, r, pos, val);

    tree[no] = tree[2 * no] + tree[2 * no + 1];
}

ll query(int no, int l, int r, int ql, int qr) {
    if (l > qr || r < ql) return 0;
    if (l >= ql && r <= qr) return tree[no];

    int m = l + (r - l) / 2;
    return m;
}
};

```

3.8 Segment Tree dinâmica

3.9 Segment Tree dinâmica 2D

3.10 Segment Tree persistente

3.11 Segment Tree temporal

3.12 Treap (BST aleatória)

4 Grafos

4.1 Ordenação Topológica

```

//  $O(V + E)$ 
vector<vector<int>> graph;
vector<int> visited, processing, order;
bool hasCycle = false;

void dfs(int u, int p) {
    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (v == p) continue; // necessario em grafo nao-
            direcionado;
        if (processing[v]) {
            hasCycle = true;
            return;
        }
        if (!visited[v])
            dfs(v);
    }

    processing[u] = false;
    order.push_back(u);
}

```

```
// Detecta e imprime qlqr ciclo encontrado
vector<int> cycle;
bool buscarCiclo(int u) {
    if (processing[u] && cycle.empty()) {
        cycle.push_back(u);
        return cycle.size() == 1;
    }
    if (visited[u]) return false;

    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (buscarCiclo(v)) {
            cycle.push_back(u);
            return cycle.back() != cycle.front();
        }
    }

    processing[u] = false;

    return false;
}

int main() {
    for (int i = 1; i <= n; ++i)
        if (dfs(i)) break; // para de procurar ciclos se
                           // achar um

    if (cycle.size()) { // imprime ciclo achado
        cout << cycle.size() << '\n';
        reverse(cycle.begin(), cycle.end());

        for (auto& i : cycle)
            cout << i << ' ';
        cout << '\n';
    }

    return 0;
}
```

4.2 Dijkstra

```
//  $O((V + E)\log V)$  ou  $O(E \log V) - S(V)$ 
const ll N = 2e5 + 1, INF = 1e18;
vector<vector<pair<ll, ll>>> graph;
vector<ll> dist;
int n, m;

void dijkstra(int s) {
    dist.assign(n + 1, INF);
    priority_queue<pair<ll, ll>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (-du > d[u]) continue; // otimização

        // v = vizinho, w = peso
        for (auto [v, w] : graph[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({-d[v], v}); // -d[v] p manter
                                     // ordenação crescente
            }
        }
    }
}
```

4.3 Bellman-Ford

```
// Calcula a menor distancia
// entre a e todos os vertices a
// detecta ciclo negativo
// Retorna 1 se ha ciclo negativo
// Nao precisa representar o grafo,
// soh armazenar as arestas
//
// Para achar ciclo positivo basta
// negar todos os pesos, logo
```

```
// um ciclo positivo vira negativo
//
// Para caminho máximo:
// Inicializar dist com -INF
// e funcao agora detecta ciclo
// positivo na n-ésima iteração
//
//  $O(V * E)$ 

const ll INF = 1e18; // 1e9 para int
struct Edge {
    int a, b, c;
};
vector<Edge> arestas;
vector<ll> dist, pai; // pai[i] = nó anterior a i
int n, m;

bool bellman_ford(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    dist[s] = 0;

    int ultimo_relaxado;
    for (int i = 0; i < n; ++i) {
        ultimo_relaxado = -1;
        for (Edge& aresta : arestas) {
            if (dist[aresta.a] < INF && dist[aresta.b] > dist
                [aresta.a] + aresta.c) {
                dist[aresta.b] = dist[aresta.a] + aresta.c;
                pai[aresta.b] = aresta.a;
                ultimo_relaxado = aresta.b;
            }
        }
    }

    return ultimo_relaxado != -1; // true se tem ciclo
    // negativo

// Retorna true se existe qualquer ciclo negativo
bool acharCicloNegativo() {
    dist.assign(n+1, 0);
    pai.assign(n+1, -1);

    ll ciclo;
    for (int i = 0; i < n; ++i) {
        ciclo = -1;
        for (Edge& a : arestas) {
            if (dist[a.a] < INF && dist[a.b] > dist[a.a] + a.
                c) {
                dist[a.b] = dist[a.a] + a.c;
                pai[a.b] = a.a;
                ciclo = a.b;
            }
        }
    }

    if (ciclo == -1) return false; // não tem ciclo
    // negativo

    for (int i = 0; i < n; ++i) ciclo = pai[ciclo];

    stack<ll> caminho;
    for (ll atual = ciclo; atual = pai[atual]) {
        caminho.push(atual);
        if (atual == ciclo && caminho.size() > 1)
            break;
    }
    while (!caminho.empty()) {
        cout << caminho.top() << ' ';
        caminho.pop();
    }
    cout << '\n';

    return true;
}
```

4.4 SPFA

4.5 Floyd-Warshall

```
// O(V^3); S(V^2)
int n, m;
void floyd_warshall(vector<vector<ll>>& d){
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
// no main:
vector<vector<ll>> d(n+1, vector<ll>(n+1, INF));
for (int i = 1; i <= n; ++i)
    d[i][i] = 0; // dist = 0 para auto-Laço
```

4.6 Caminho Euleriano

4.7 Componentes Biconexos (BCC)

4.8 Componentes Fortemente Conexos (SCC)

```
// O(V + E) - Algoritmo de Kosaraju
// Encontra todos os componentes fortemente conexos em
// grafo direcionado
```

```
const int MAXN = 1e5+1;
vector<vector<int>> grafo, grafoReverso;
vector<int> ordem; // Ordem topológica
bitset<MAXN> visitado;
int n, m; // vértices, arestas
```

```
// ordenação topológica
```

```
void dfsOrdem(int u) {
    visitado[u] = true;
    for (auto& v : grafo[u])
        if (!visitado[v])
            dfsOrdem(v);
    ordem.push_back(u);
}
```

```
// DFS no grafo reverso para encontrar SCCs
```

```
void dfsComponente(int u) {
    visitado[u] = true;
    for (auto& v : grafoReverso[u])
        if (!visitado[v])
            dfsComponente(v);
}
```

```
// Retorna vetor com um representante de cada
// componente
```

```
vector<int> kosaraju() {
    for (int i = 1; i <= n; ++i)
        if (!visitado[i])
            dfsOrdem(i);
    reverse(ordem.begin(), ordem.end());
```

```
visitado.reset();
vector<int> componentes;
```

```
for (auto& u : ordem) {
    if (!visitado[u]) {
        dfsComponente(u);
        componentes.push_back(u);
    }
}
```

```
return componentes;
```

4.9 Árvore Geradora Mínima (MST)

```
// Prim
// O((V + E) * LogV)
```

```
const int MAXN = 1e5+1;
bitset<MAXN> visitado;
```

```
int prim() {
    priority_queue<pair<int,int>> pq; // {peso, nó}
    pq.emplace(0,1); // começa no nó 1
```

```
int sum = 0;
while (!pq.empty()) {
    auto [w, u] = pq.top();
    pq.pop();

    if (visitado[u]) continue;

    sum += -w;
    visitado[u] = true;

    for (auto& [v,w] : adj[u]) // adj = {nó, peso}
        if (!visitado[v])
            pq.emplace(-w, v);
}

return visitado[n]? sum : -1;
}
```

```
// Kruskal (usa Union-find)
```

```
// O(E * LogE)
```

```
// N precisa guardar grafo, soh arestas
```

```
struct Edge {
    int a,b, c;
};
vector<Edge> arestas;
vector<int> p;
```

```
int kruskal() {
    for (int i = 1; i <= n; ++i) p[i] = i;
    sort(arestas.begin(), arestas.end(), [](auto a, auto
        b){
            return a.c < b.c;
        });
}
```

```
int sum = 0;
for (auto& e : arestas) {
    if (find(e.a) != find(e.b)) {
        join(e.a, e.b);
        sum += e.c;
    }
}
```

```
return find(1)==find(n)? sum : -1;
```

```
}
```

```
int find(int x) {
    if (x == p[x]) return x;
    return p[x] = find(p[x]);
}
```

```
void join(int x, int y) {
    p[find(x)] = find(y);
}
```

4.10 Clique Maximal

4.11 Clique Máximo

4.12 Ciclo de Média Mínima

4.13 AGM Manhattan

5 Programação Dinâmica

5.1 PD em dígitos

5.2 SOS DP

6 Matemática

6.1 Fórmulas

```
// Soma de Números Consecutivos
```

```
// Soma= (Primeiro + Último) * Quantidade de Termos *
// 1/2
```

6.2 Exponenciação Binária

```
// a^b O(LogN)
ll exp(ll a, ll b, const ll M = 1e9 + 7) {
    a %= M;
    ll res = 1;

    while (b) {
        if (b % 2) res = res * a % M;
        a = a * a % M;
        b /= 2;
    }
    return res;
}
```

6.3 Soma e multiplicação com __int128

6.4 Primos

```
mashu lucky prime : 91145149
1097774749, 1076767633, 100102021, 999997771
1001010013, 1000512343, 987654361, 999991231
999888733, 98789101, 987777733, 999991921, 1010101333
```

6.5 Pares coprimos

6.6 Exponenciação rápida

6.7 Exponenciação rápida de matrizes

6.8 Função totiente (ϕ)

6.9 Tabela de fatores

6.10 Números de Catalan

6.11 Teste de primalidade Miller-Rabin

6.12 Fatoração Pollard-Rho

6.13 Fatoração prima em $O(\log n)$

6.14 Multiplicação $O(1)$ com __int128

6.15 Problema de Josefo (Josephus)

6.16 Soma harmônica

6.17 Contagem de Pólya

6.18 FFT (Transformada Rápida de Fourier)

6.19 Jogo de Grundy

6.20 Algoritmo de Euclides Estendido

6.21 Teorema Chinês do Resto (CRT)

7 Strings

7.1 Arranjo de Sufixos (SA)

7.2 KMP (Knuth-Morris-Pratt)

```
// p[i] = comprimento do maior prefixo próprio do
// padrão s que também é sufixo
// do texto t[0..i] Prefixo/Sufixo próprio é um prefixo
// /sufixo que não é a
// string inteira Usos:
// - Busca de padrão único (igual Z, mas mais natural
// para busca online)
// - Busca online/streaming: processa t caractere a
// caractere sem ter t
// completo
// - Menor período de string: n - lps[n-1] é o
// período mínimo
// - Autômato de string: transformar lps em DFA para
// múltiplas consultas no
// mesmo padrão
```

```
// O(m), m = s.size()
vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[j] != s[i]) j = p[j - 1];
        if (s[j] == s[i]) j++;
        p[i] = j;
    }
    return p;
}
```

```
// O(n + m) KMP: busca padrao s em texto t, retorna
// índices de cada ocorrência
// (0-based)
```

```
vector<int> kmp(string &t, string &s) {
    vector<int> p = pi(s + '$'), match;
    for (int i = 0, j = 0; i < t.size(); ++i) {
        while (j > 0 && s[j] != t[i]) j = p[j - 1];
        if (s[j] == t[i]) j++;
        if (j == s.size()) match.push_back(i - j + 1);
    }
    return match;
}
```

7.3 Hash simples

7.4 Hash duplo

7.5 Rabin-Karp

```
// O(n + m) médio, O(nm) pior caso
// Busca de padrão p em texto t usando hashing de
// janela deslizante
// Usos:
// - Busca de múltiplos padrões simultaneamente (
// coloca todos num set de
// hashes)
// - Detectar strings repetidas / substrings
// duplicadas (+busca binária)
//
// Atenção: colisões são raras mas possíveis – confirme
// com strcmp se necessário
// Use double hashing (dois módulos) para competições
// com anti-hash tests
```

```
const ll MOD = 1e9 + 7, BASE = 31;
```

```
vector<int> rabin_karp(string t, string p) {
    int n = t.size(), m = p.size();
    vector<int> ocorrencias;
```

```
// Pré-computa potências de BASE
vector<ll> pw(max(n, m) + 1);
pw[0] = 1;
for (int i = 1; i <= max(n, m); i++) pw[i] = pw[i -
1] * BASE % MOD;
```

```
// Hash do padrão
ll hp = 0;
for (int i = 0; i < m; i++) hp = (hp + (p[i] - 'a' + 1) * pw[i]) % MOD;

// Hash prefixo do texto: h[i] = hash(t[0..i-1])
vector<ll> h(n + 1, 0);
for (int i = 0; i < n; i++) h[i + 1] = (h[i] + (t[i] - 'a' + 1) * pw[i]) % MOD;

// Hash de t[L..r-1] = (h[r] - h[L]) / pw[L] * inv(pw[L])
// Evita inversão modular: compara (h[r] - h[L]) com hp * pw[L]
for (int i = 0; i + m <= n; i++) {
    ll ht = (h[i + m] - h[i] + MOD) % MOD;
    if (ht == hp * pw[i] % MOD) ocorrencias.push_back(i);
}

return ocorrencias; // índices (0-based) onde p ocorre em t
}
```

7.6 Trie

7.7 Função Z

```
// O(n)
// z[i] = comprimento do maior prefixo de s que também é prefixo de s[i..]
// Usos:
// - Busca de padrão: z(p + "#" + t), ocorrências onde z[i] == |p|
// - Menor período de string: menor k tal que k | n e z[k] == n - k
// - Contar prefixos distintos que ocorrem em s: acumular z[i] com cuidado
// - Comparar sufixos rapidamente sem suffix array
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;

        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

7.8 Rotação mínima

7.9 Manacher (palíndromos)

7.10 Árvore de palíndromos (PalTree)

7.11 Subsequências distintas

8 Árvores

8.1 LCA (Menor Ancestral Comum)

8.2 Hash de árvore

8.3 Decomposição Pesada-Leve (HLD)

9 Geometria

9.1 Definições 2D

9.2 Definições 3D

9.3 Definição de reta

9.4 Operações básicas

9.5 Área do polígono

9.6 Ponto dentro do polígono

9.7 Fecho convexo

9.8 Soma de Minkowski

9.9 Distância mais curta entre polígonos

9.10 Truque do fecho convexo (CHT)

9.11 Ordenação polar

9.12 Teorema de Pick

9.13 Par de pontos mais próximos

9.14 Par de pontos mais distantes

9.15 Mediana geométrica (Weiszfeld)

9.16 Linha de varredura

9.17 Interseção círculo-polígono

9.18 Tangentes a dois círculos

9.19 Interseção de dois círculos

9.20 Cobertura de círculo

9.21 Círculo mínimo envolvente

9.22 Esfera mínima envolvente

9.23 Retângulo envolvente de área mínima/máxima

9.24 Interseção de semiplanos

9.25 União de polígonos

9.26 Cobertura de polígonos

9.27 Pontos notáveis do triângulo

10 Problemas Especiais

10.1 Contagem de substrings

10.2 Ordem parcial 3D

10.3 LCS cíclico

10.4 Simulated Annealing (têmpera simulada)

10.5 Raiz quadrada discreta

Papel milimetrado para rascunho



